Semana 4. Interrupciones externas en ESP32

1. Sondeo e interrupciones

Con **sondeo (polling)**, loop() revisa repetidamente el pin. Si el programa está ocupado, puede pasar por alto un toque corto. Con una **interrupción externa**, el hardware detecta el cambio y la CPU ejecuta brevemente una **ISR** (rutina de servicio de interrupción) antes de continuar.

2. Flancos y configuración del pulsador

Un **flanco** es el cambio de nivel de una señal. En un pulsador con INPUT_PULLUP, el pin normalmente está en HIGH y baja a LOW al presionar.

Modo	Transición	Con INPUT_PULLUP
FALLING	HIGH → LOW	Al presionar
RISING	LOW → HIGH	Al soltar
CHANGE	Ambas	Al presionar y soltar

En la práctica se usa **GPIO21** como entrada con pull-up. Los GPIO 34–39 funcionan solo como entrada y no tienen pull-up interna; se evitan los GPIO 6–11 y los pines que afectan el arranque.

3. attachInterrupt() e ISR

La interrupción se configura en **setup()** indicando el pin, la función que se ejecutará y el flanco. La función se escribe **sin paréntesis** dentro de attachInterrupt().

```
pinMode(21, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(21),
                isrBoton, FALLING);
```

La ISR debe ser breve: no se recomienda utilizar dentro **delay()**, **Serial.println()** ni procesos largos. Una rutina que se bloquea puede provocar fallas y reinicios por el watchdog.

4. IRAM_ATTR, volatile y patrón bandera

IRAM_ATTR sitúa la ISR en memoria interna de instrucciones para que pueda atender eventos cuando el acceso a flash no esté disponible. **volatile** evita que el compilador dé por fijo un valor que puede cambiar desde una ISR; no vuelve atómicas todas las operaciones.

```
volatile bool alerta = false;
void IRAM_ATTR isrBoton() {
  alerta = true;
}
void loop() {
  if (alerta) {
    alerta = false;
    Serial.println("Alerta detectada");
  }
}
```

Así la ISR únicamente marca la bandera y **loop()** realiza la acción. Si llegan varias pulsaciones antes de revisar la bandera, pueden quedar reunidas en un solo aviso.

5. Rebote y antirrebote

Un botón mecánico produce varios cambios de nivel en pocos milisegundos durante una pulsación. Por eso una sola presión puede generar varias interrupciones. El ejemplo de clase utiliza **50 ms** como intervalo mínimo entre eventos válidos.

```
const uint32_t T_REBOTE = 50;
volatile bool alerta = false;
uint32_t tUltimo = 0;
void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  if (ahora - tUltimo >= T_REBOTE) {
    alerta = true;
    tUltimo = ahora;
  }
}
```

Este fragmento se completa en setup() con la configuración del botón y attachInterrupt(), y en loop() con el procesamiento de la bandera. La resta de tiempos permite comprobar el intervalo de antirrebote.

6. Ejercicios y preguntas resueltas

1. Botón con pull-up: ¿qué flanco detecta la pulsación? **FALLING**, porque pasa de HIGH a LOW.

2. Si la ISR cambia una variable que loop() lee, ¿qué se usa? **volatile**. Si tUltimo solo se utiliza dentro de la ISR, no necesita volatile por ese motivo.

3. ¿Qué pasa si se imprimen mensajes desde la ISR? La operación puede tardar o bloquear. Mejor marcar una **bandera** y mostrar el mensaje en loop().

4. ¿Diez pulsaciones generan exactamente diez interrupciones? No necesariamente: el **rebote** puede producir varios flancos por cada pulsación.

5. Si loop() está ocupado, la ISR puede marcar la bandera y loop() la atenderá después. La bandera no cuenta varios avisos por separado.